Introduction to Hadoop

Big Data

본 자료에는 네이버에서 제공한 나눔글꼴이 적용되어 있습니다.

Learning Objectives

❖ 학습내용

- Hadoop

❖ 학습목표

- Hadoop 프레임워크의 기능과 역사를 알아본다.

❖ Hadoop 이란?

- " an open source software platform for distributed storage and distributed processing of very large data sets on computer clusters built from commodity hardware"
  –Hortonworks

❖ Hadoop 이란?

- Software platform: 하나의 컴퓨터가 아닌 컴퓨터 클러스터에 의해 작동하는 software 묶음
- Distributed storage: 하나의 저장소에 국한되지 않음, 클러스터에 컴퓨터를 추가하면 해당 컴퓨터의 HDD(SSD)가 데이터 저장소 일부가 됨
  - 클러스터의 모든 저장소에 분산된 데이터를 단일 파일 시스템으로 보여줌
  - 데이터의 백업 복사본을 클러스터의 다른 컴퓨터에도 보관함(소실 데이터 복구)

❖ Hadoop 이란?

- Distributed processing: 클러스터에 속한 컴퓨터들이 함께 데이터를 처리함 (병렬 처리)
  - Hadoop은 클러스터 내 모든 컴퓨터 CPU에게 작업을 분배함

❖ Hadoop의 역사

- Google의 GFS(google file system)과 MapReduce 발표 (2003-2004)
- Yahoo의 오픈소스 웹 검색 엔진 'Nutch'
  - Nutch에 MapReduce를 구현 (2005)
  - Hadoop 프로젝트 (Doug Cutting과 Tom White) (2006)
- Hadoop: High-Availability Distributed Object-Oriented Platform

Why Hadoop?

❖ Hadoop의 장점

- 확장성
- 내결함성과 안정성
- 효율성
- 다양한 데이터 유형 지원
- 다양한 도구와 프레임워크 지원

Summary

- Hadoop

  대용량 데이터를 분산 저장하고 분산 처리하는
  오픈소스 소프트웨어 플랫폼

Hadoop Ecosystem

Big Data

본 자료에는 네이버에서 제공한 나눔글꼴이 적용되어 있습니다.

Learning Objectives

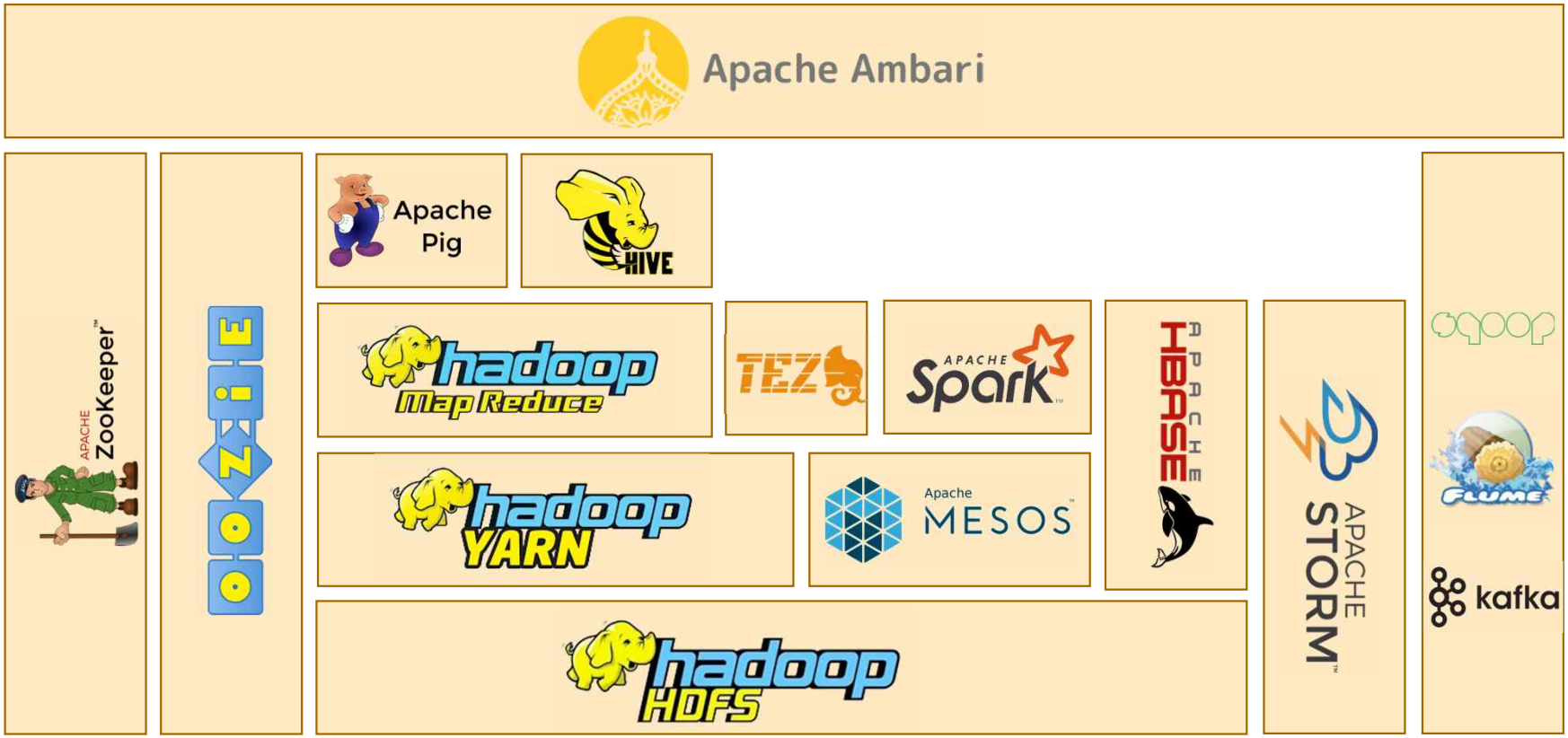
❖ 학습내용

- Hadoop ecosystem

❖ 학습목표

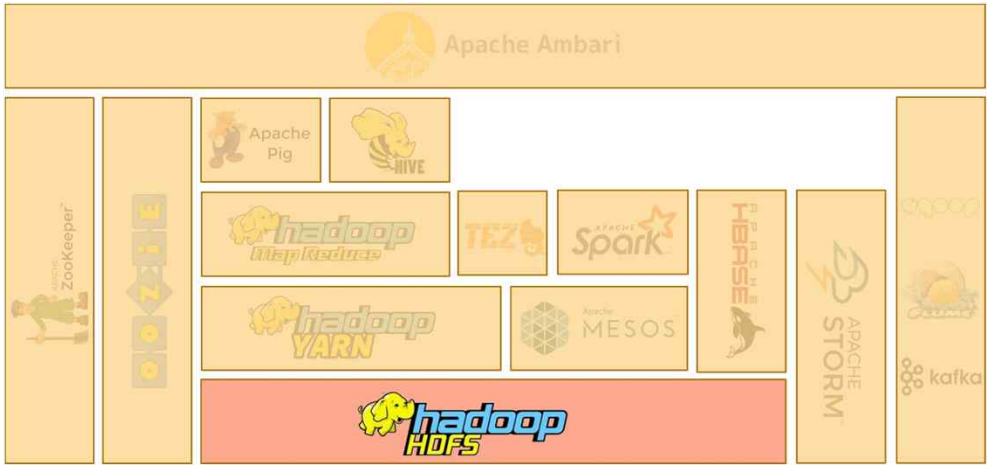
- Hadoop ecosystem의 구성에 대해 알아본다.

❖ Hadoop 생태계


Apache ZooKeeper
OOZIE
Apache Ambari
Apache Pig
HIVE
hadoop Map Reduce
TEZ
APACHE Spark
hadoop YARN
Apache MESOS
APACHE HBASE
hadoop HDFS
APACHE STORM
sqoop
FLUME
kafka

Hadoop Ecosystem

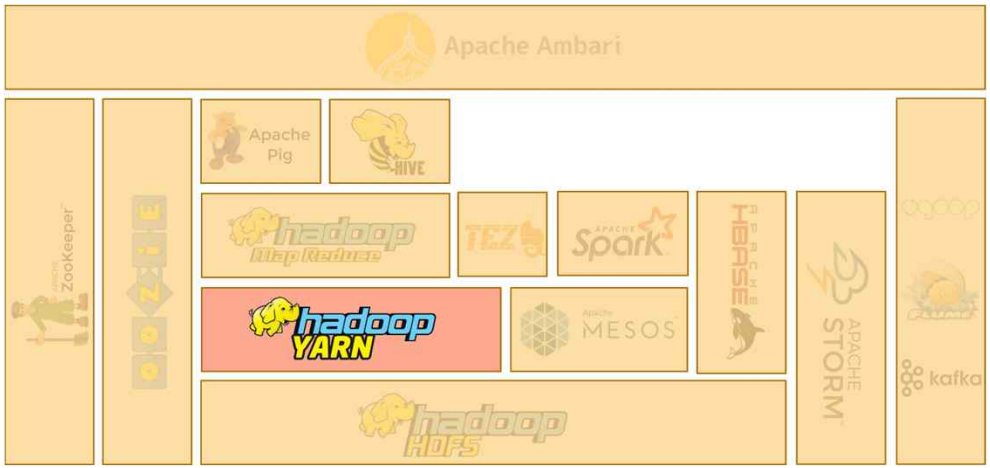
❖ HDFS



- Hadoop distributed file system
- Hadoop의 분산 데이터 저장소
- 빅데이터를 하둡 클러스터의 컴퓨터들에 분산 저장하는 시스템
  (클러스터의 하드드라이브를 하나의 거대한 파일 시스템처럼 사용)
- 데이터의 여분 복사본을 생성

Hadoop Ecosystem

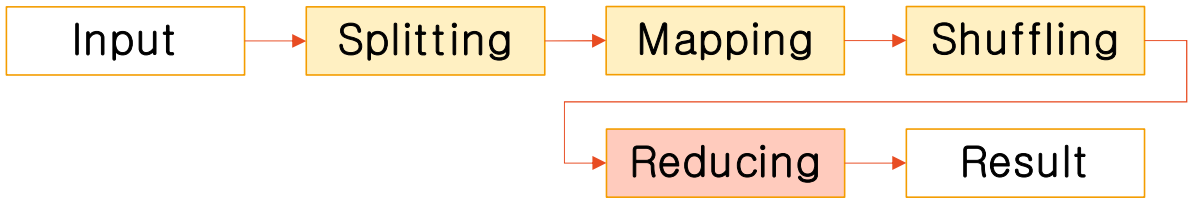
❖ YARN



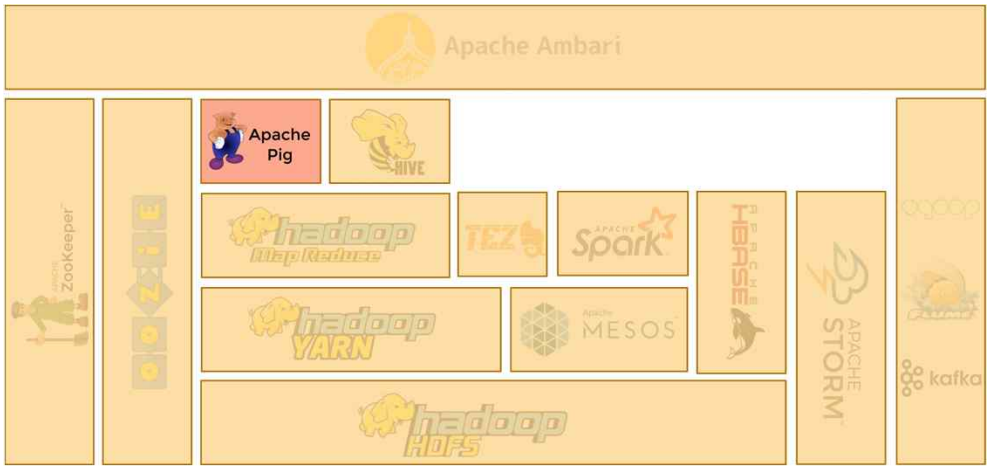
- Yet Another Resource Negotiator (또 다른 리소스 교섭자)
- Hadoop의 클러스터 자원 관리
  - 어떠한 노드가 작업을 언제 실행하고, 어떠한 노드가 추가 작업을 할 수 있는지 없는지를 결정함

❖ MapReduce

- Mapper와 reducer로 구성됨 (2개의 구분된 스크립트 or 함수)
- Mapper: 클러스터에 분산된 데이터를 효율적으로 동시에 변형
- Reducer: 변형된 데이터를 집계



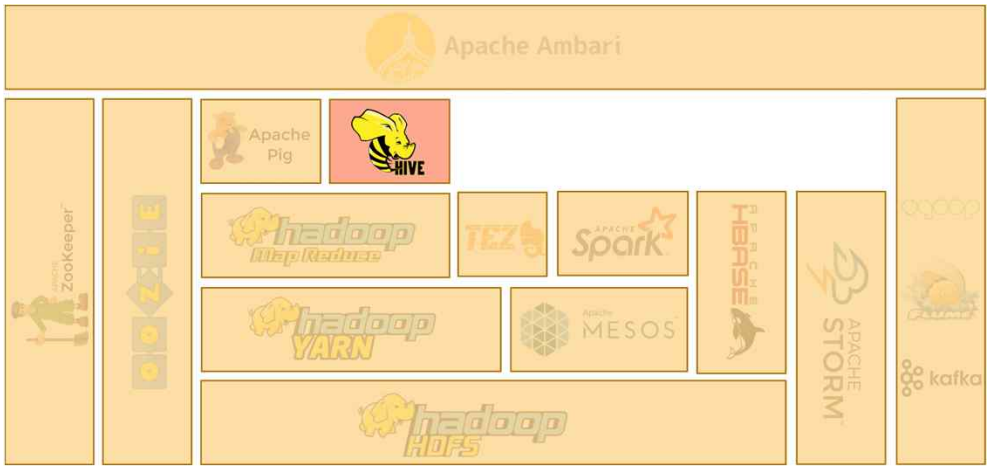
❖ Pig



- MapReduce 위의 고수준 스크립팅 언어
- MapReduce 프로그래밍 대신 SQL과 유사한 Pig Latin 자체 스크립트 언어로 처리
- Pig Latin을 해석하여 MapReduce로 변환하여 실행

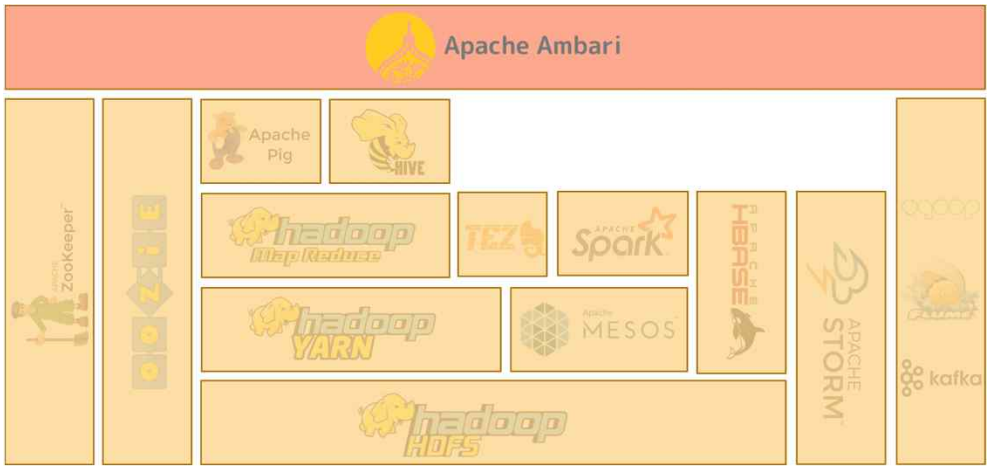
Hadoop Ecosystem

❖ Hive



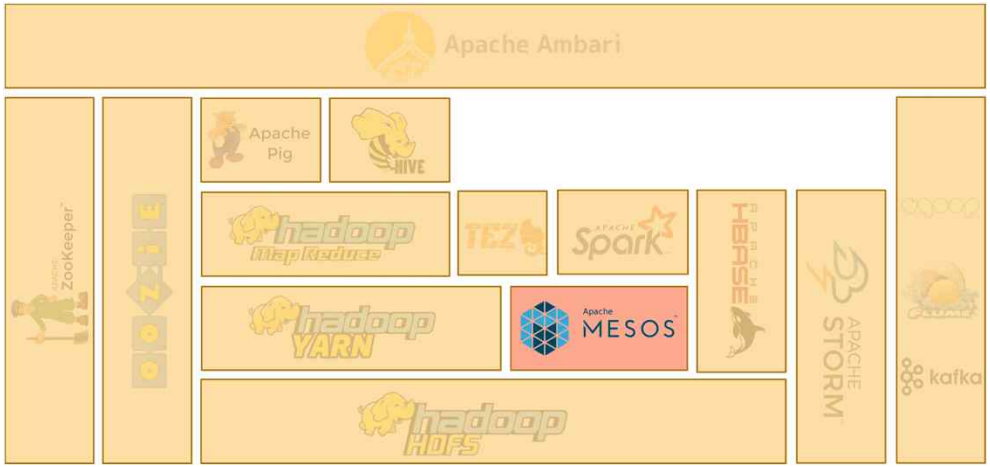
- Hadoop에서 동작하는 Data warehouse
- 데이터 요약, 쿼리 및 분석 기능 제공
- ODBC(Open Database Connectivity)를 통해 데이터베이스에 접속
- 관계형 데이터베이스가 아니더라도, SQL과 유사한 HiveQL로 쿼리함
- MapReduce의 모든 기능을 지원

❖ Ambari



- Hadoop 클러스터의 관리와 모니터링을 간소화함
- 사용하기 쉬운 UI와 REST API를 통해 구현
- 클러스터를 모니터링하고 구성을 변경하는데 사용됨

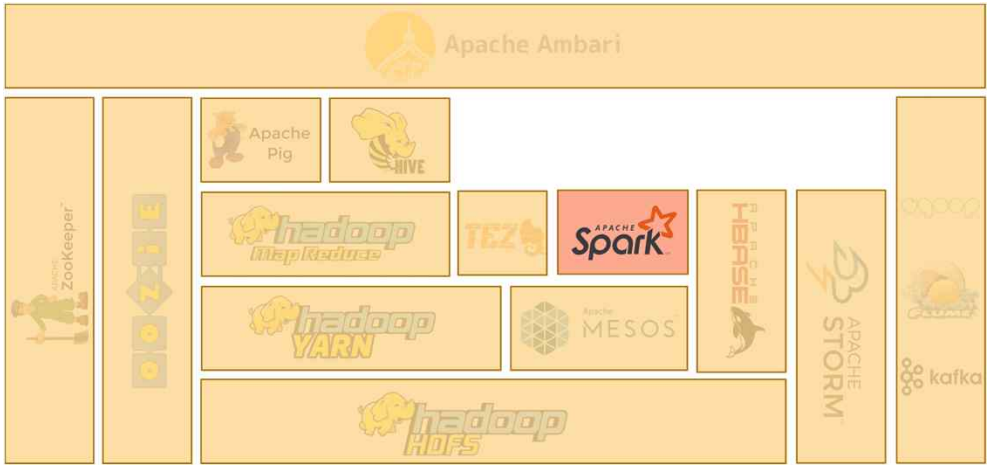
❖ Mesos



- 클라우드 인프라스트럭쳐 및 컴퓨팅 엔진의 다양한 자원을 통합적으로 관리할 수 있도록 만든 자원 관리 프로젝트 (Hadoop 외 다양한 애플리케이션에서 활용)
- YARN과 동일한 리소스 관리 기능 수행

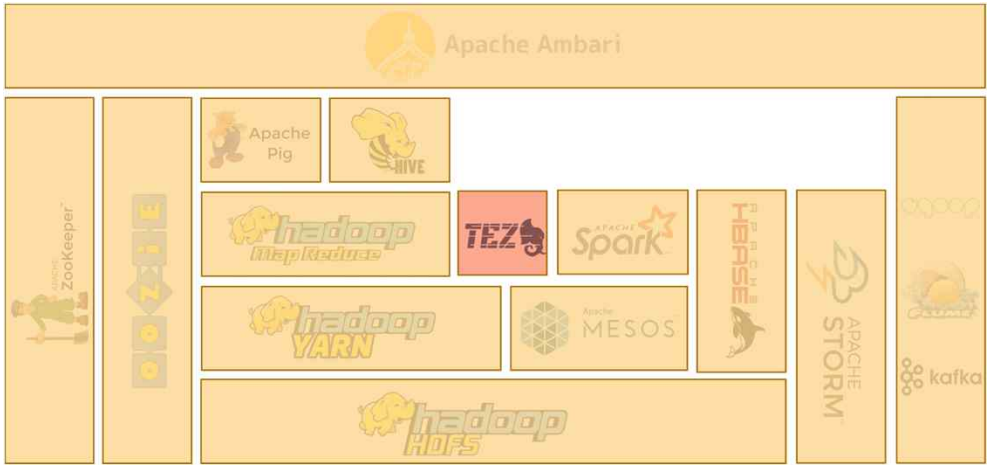
Hadoop Ecosystem

❖ Spark



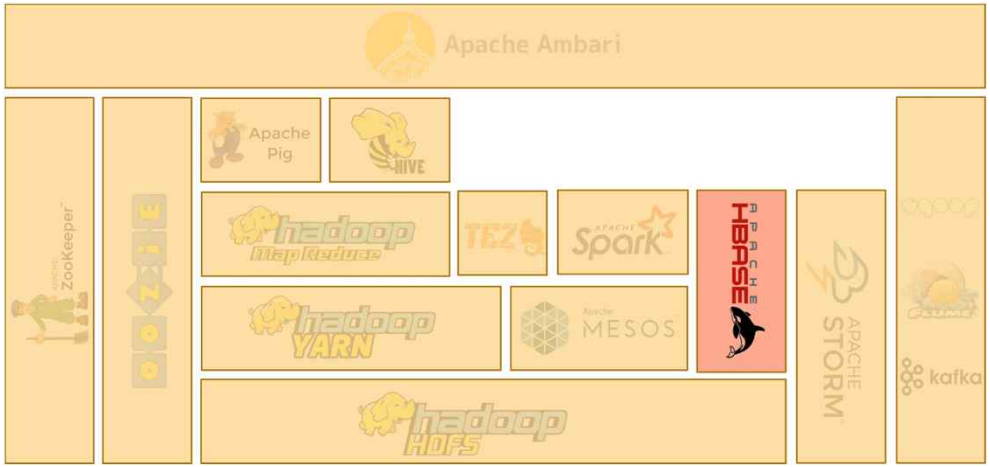
- 클러스터 환경에서 데이터를 병렬로 처리하는 라이브러리 집합
- 데이터를 신속하고 효율적, 안정적으로 처리
- Python, Java, Scala, R을 지원하며, SQL, 스트리밍, 머신러닝 등 넓은 범위의 라이브러리 제공

❖ TEZ



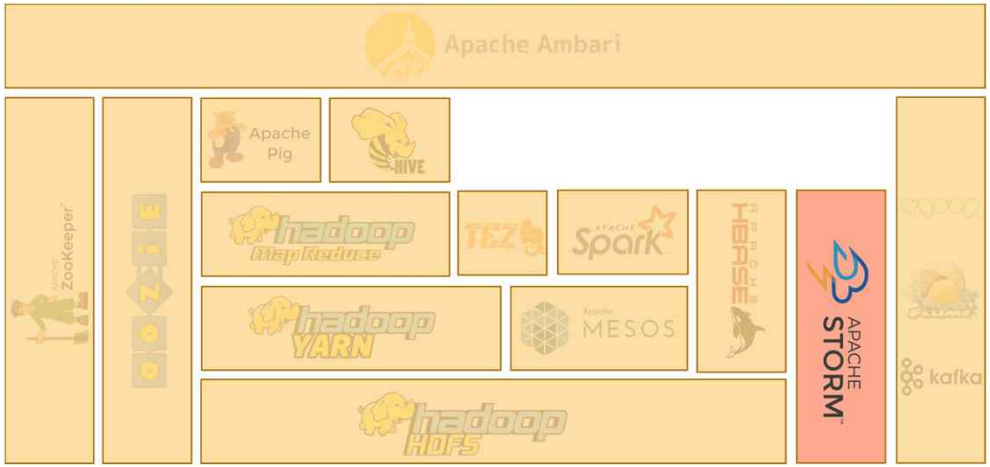
- Spark와 유사한 역할
- 방향성 비순환 그래프(DAG: Directed Acyclic Graph) 사용
  - 쿼리 실행을 더 효율적으로 할 수 있음
- 주로 Hive와 함께 사용하여 성능을 가속화함

❖ HBASE



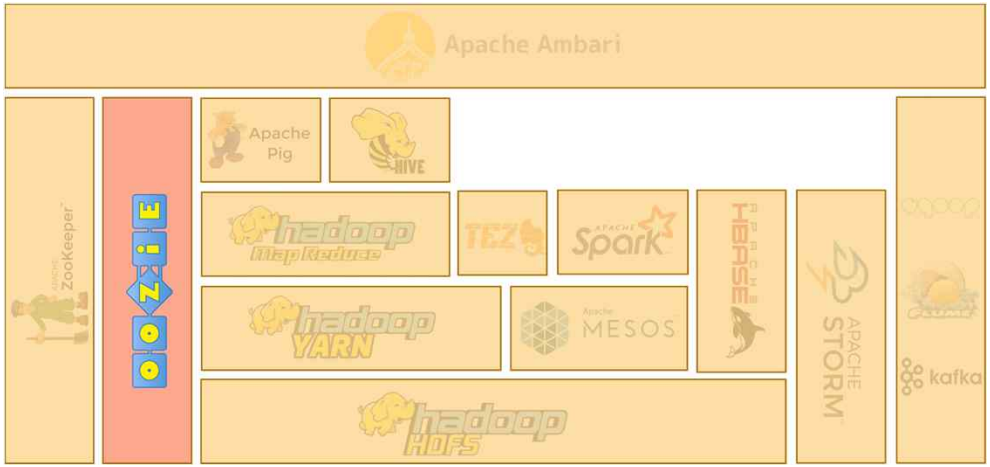
- HDFS 기반 NoSQL 데이터베이스
- 대규모 클러스터에서도 빠른 성능 유지
- HBase 테이블 기반 분석 워크로드 실행
- OLTP(Online Transaction Processing)에 적절함

❖ Storm



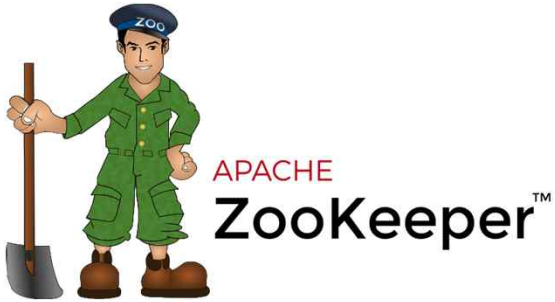
- 분산형 스트림 프로세싱 연산 프레임워크
- 스트리밍 데이터를 실시간으로 처리함
  - 센서 또는 웹로그로부터 데이트를 스트리밍 하는 경우
  - 실시간 분석, machine learning, 운영 모니터링 용도
  - Spark streaming과 동일한 역할

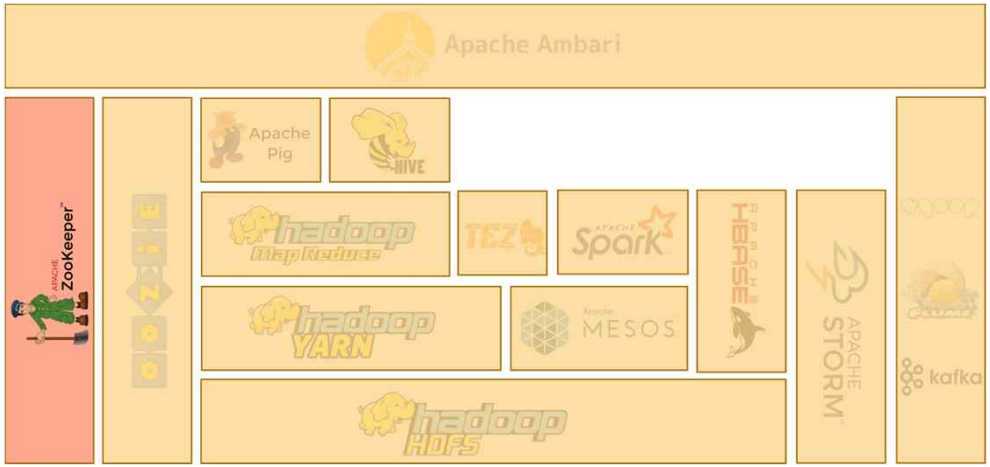
❖ Oozie



- 클러스터의 작업 스케줄링
- control node
  - 작업의 시작과 끝을 지정하고, 작업 진행 경로를 통제함
- action node
  - 하나의 워크플로우가 계산 또는 처리작업을 하도록 함
  - MapReduce, HDFS 관련 작업, Pig, SSH, HTTP, email 작업 등

❖ ZooKeeper

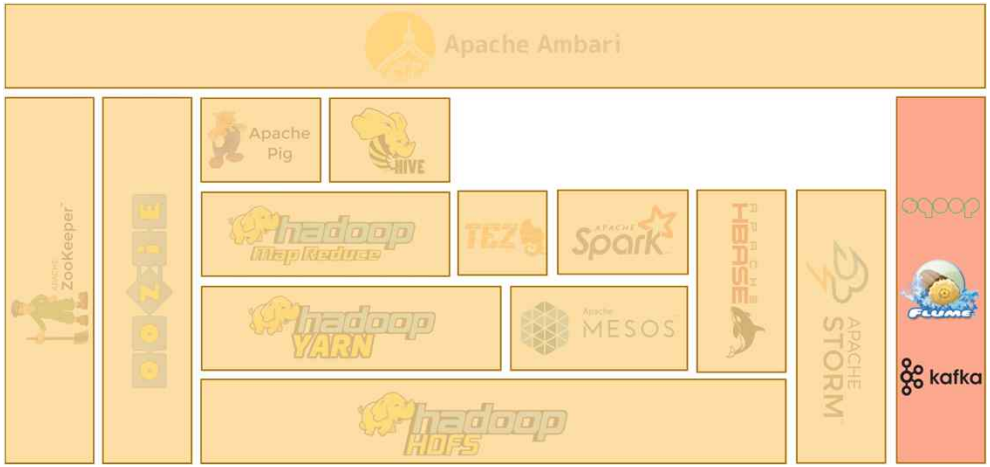




- 분산 코디네이션
  - 설정/클러스터 관리: 설정 정보, 노드 추가 제외 정보 공유
  - 리더 채택: 리더 노드 선정
  - 락, 동기화 서비스: 클러스터 전체를 동기화
- 노드 상태 점검을 통한 안정적인 클러스터 유지

Hadoop Ecosystem

❖ Sqoop, Flume, Kafka



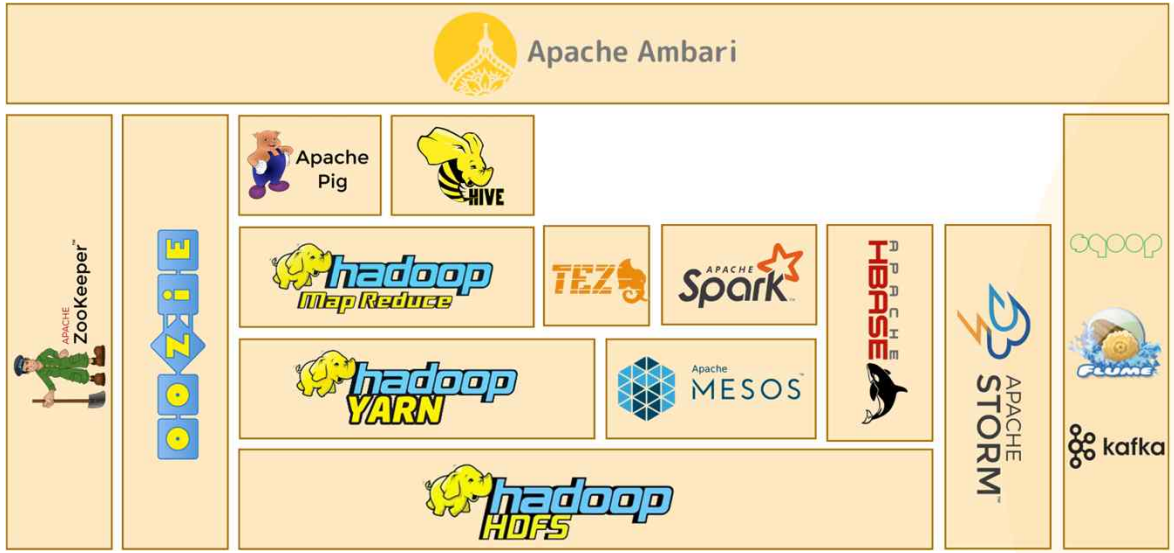
- Data injection
  - Sqoop: 데이터베이스와 Hadoop을 잇는 연결 장치
  - Flume: 실시간 데이터를 안정적으로 클러스터에 불러옴
  - Kafka: PC나 서버 클러스터에서 모든 종류의 데이터를 수집

Hadoop Ecosystem

❖ 기타

Summary

- Hadoop ecosystem



HDFS

Big Data

본 자료에는 네이버에서 제공한 나눔글꼴이 적용되어 있습니다.

Learning Objectives

❖ 학습내용

- HDFS (Hadoop Distributed File System)

❖ 학습목표

- HDFS의 구조, 특성, 파일 읽기와 쓰기 프로세스에 대해 이해한다.

❖ HDFS (Hadoop Distributed File System)

- 대용량 파일을 분산된 서버에 저장하고, 많은 클라이언트가 저장된 데이터를 빠르게 저장할 수 있게 설계된 파일 시스템
- 저사양 서버를 이용해 스토리지를 구성할 수 있음
- 물리적으로 분산된 서버의 로컬 디스크에 저장되고, 파일의 읽기와 저장과 같은 제어는 HDFS의 API를 통해 처리됨

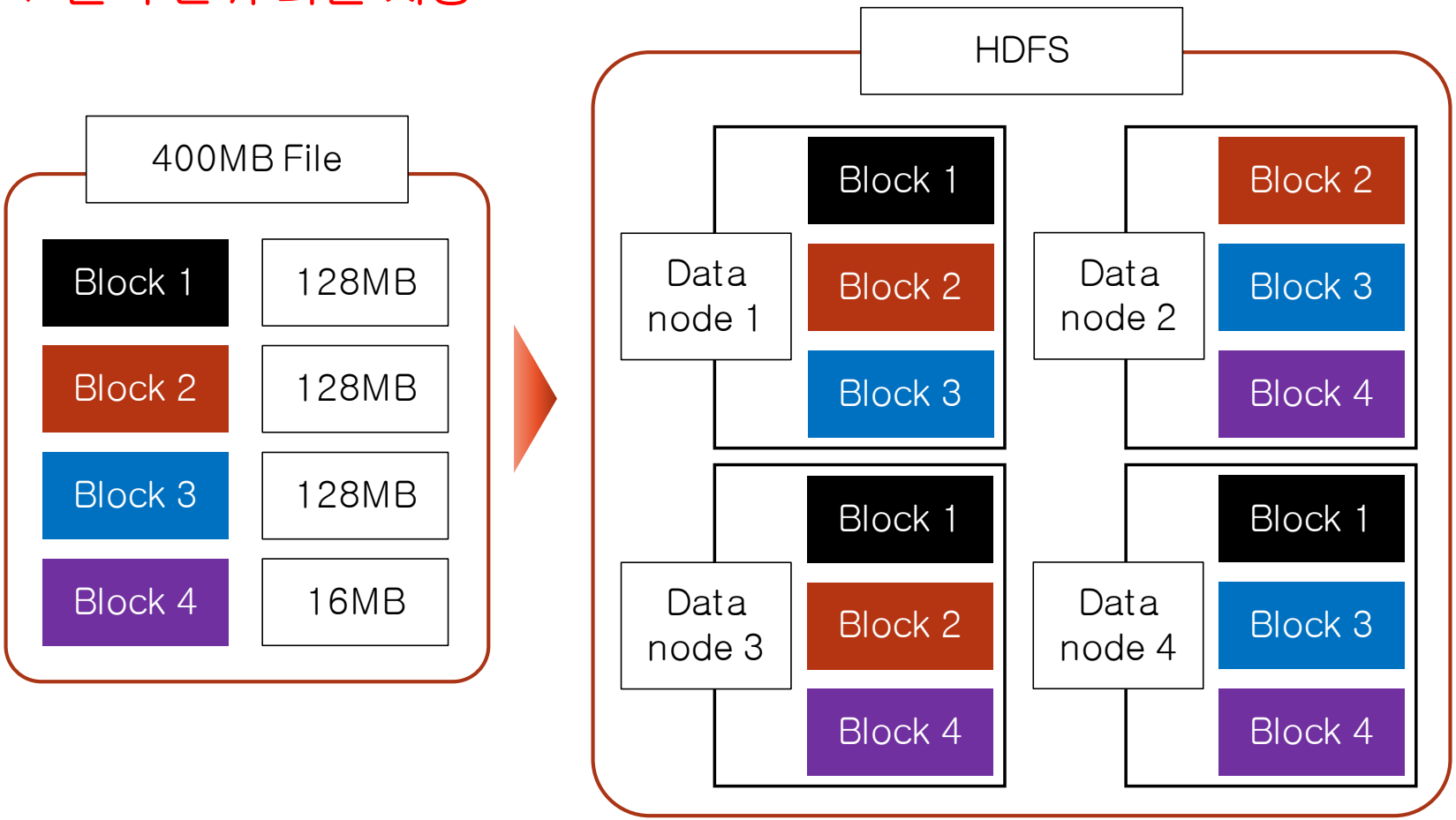
Block Structured File System

❖ 블록 사이즈

- HDFS의 기본 블록 사이즈는 일반적인 파일 시스템 또는 데이터베이스보다 큰 128MB (2.0 이전 버전의 경우, 64MB)
- Disk seek time 감소
  - 블록 사이즈를 이용하여 탐색 시간을 줄임
- 네임노드의 메타데이터 크기 감소
  - 네임노드는 블록 위치, 파일명, 권한 정보와 같은 메타데이터를 저장하고 관리하며 블록의 사이즈가 클 경우, 메타데이터의 크기가 감소함
- 클라이언트와 네임노드 간 통신 감소
  - 블록이 커짐에 따라 블록의 수가 줄어들면서 파일 접근 횟수가 감소함

Block Structured File System

❖ 블록 단위 파일 저장



Namenode and Datanode

❖ Name node

- 메타데이터 관리
  - 파일시스템을 유지하기 위한 메타데이터를 관리함
  - 메타데이터는 파일명, 디렉토리, 크기, 권한, 블록 매핑 정보 등으로 구성됨
- 데이터노드 모니터링
  - 데이터노드는 네임노드에 주기적으로 heartbeat와 block report 전송, 상태·용량 모니터링
    (heartbeat: 동작 중임을 알림, block report: 블록 위치 정보)
- 블록 관리
  - 데이터노드 장애 발생 시, 블록을 새로운 데이터노드로 복제
  - 데이터노드의 용량이 부족한 경우, 블록 이동
- 클라이언트 요청 접수
  - 클라이언트가 HDFS에 접근하기 위해서는 네임노드에 접속하여
    권한 확인 절차가 필요

Namenode and Datanode

❖ Data node

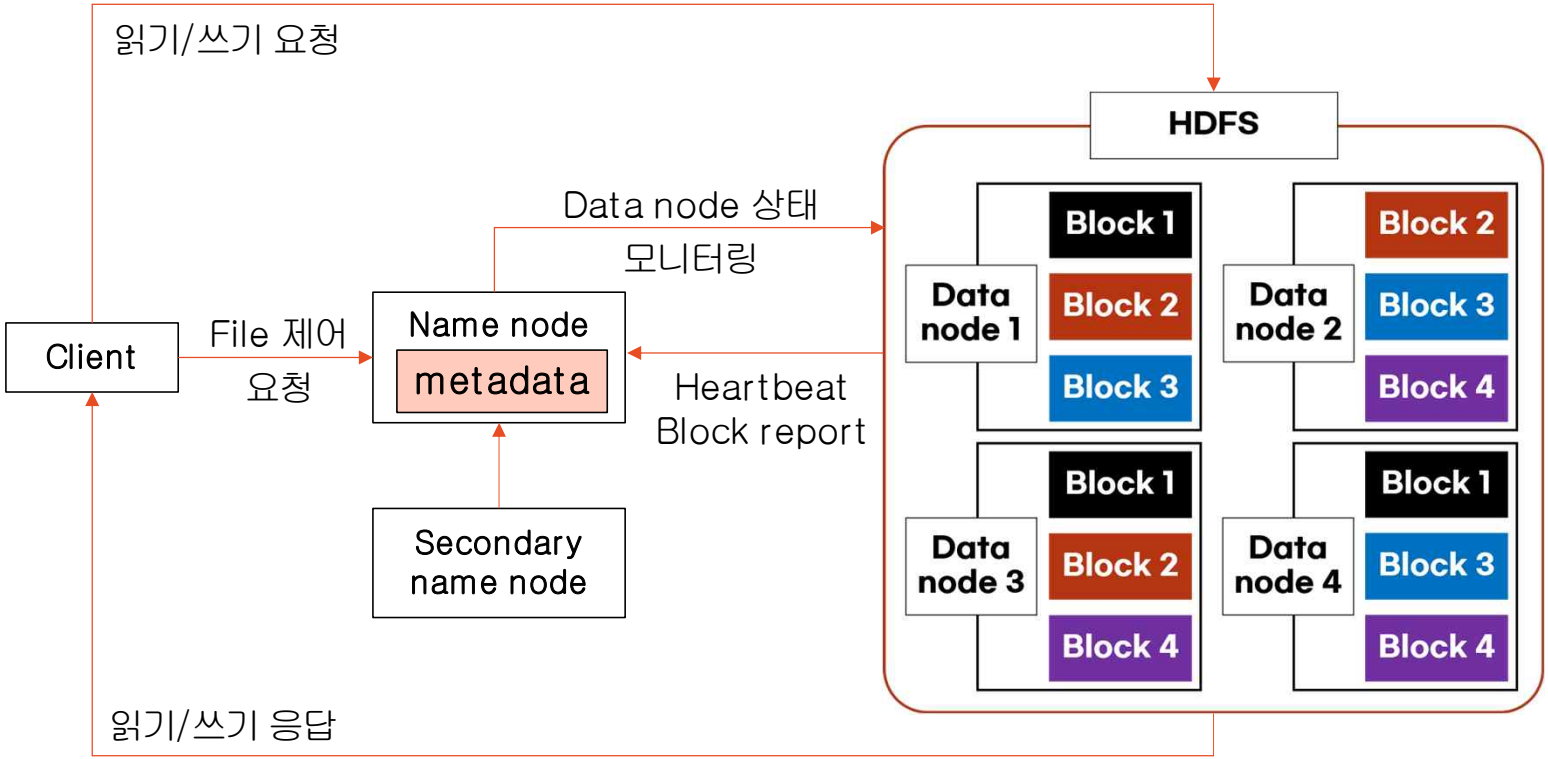
- 클라이언트가 HDFS에 저장하는 파일을 로컬디스크에 유지함

- 블록의 목록을 주기적으로 네임노드에 보고

- Name node가 Data node의 블록 구조를 관리하지만, 데이터의 읽기와 쓰기는 클라언트와 Data node에 의해 수행됨

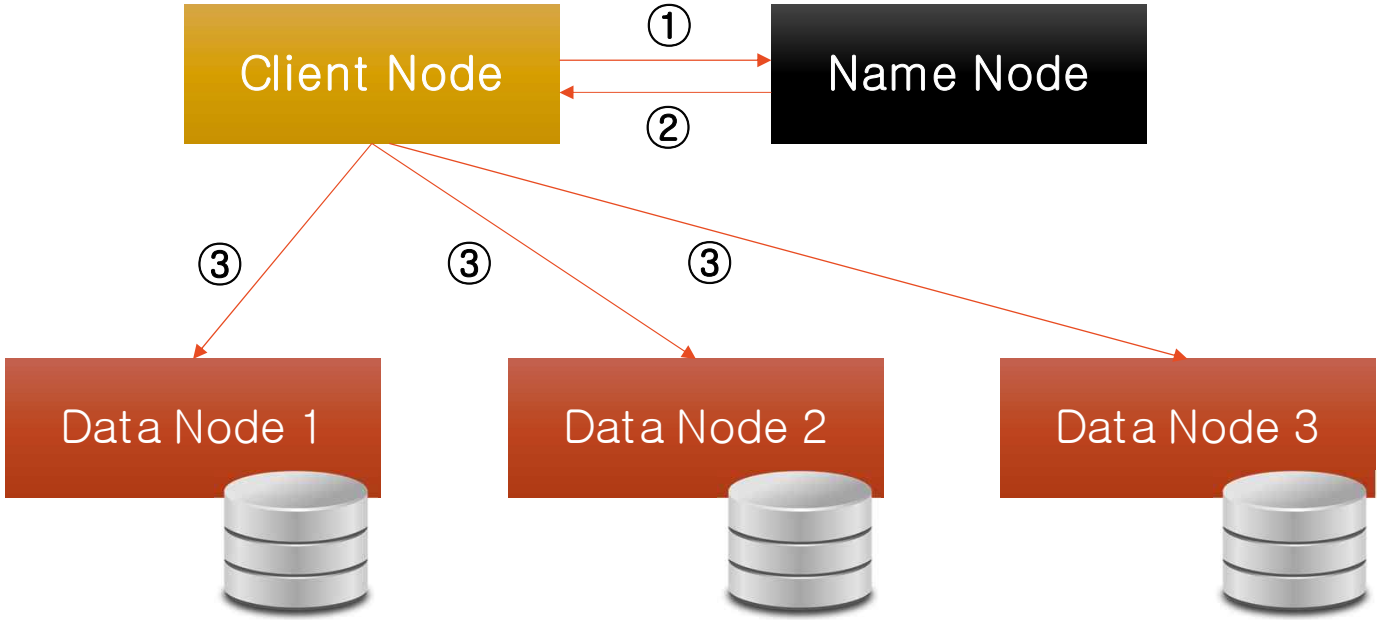
Namenode and Datanode

❖ Name node와 Data node



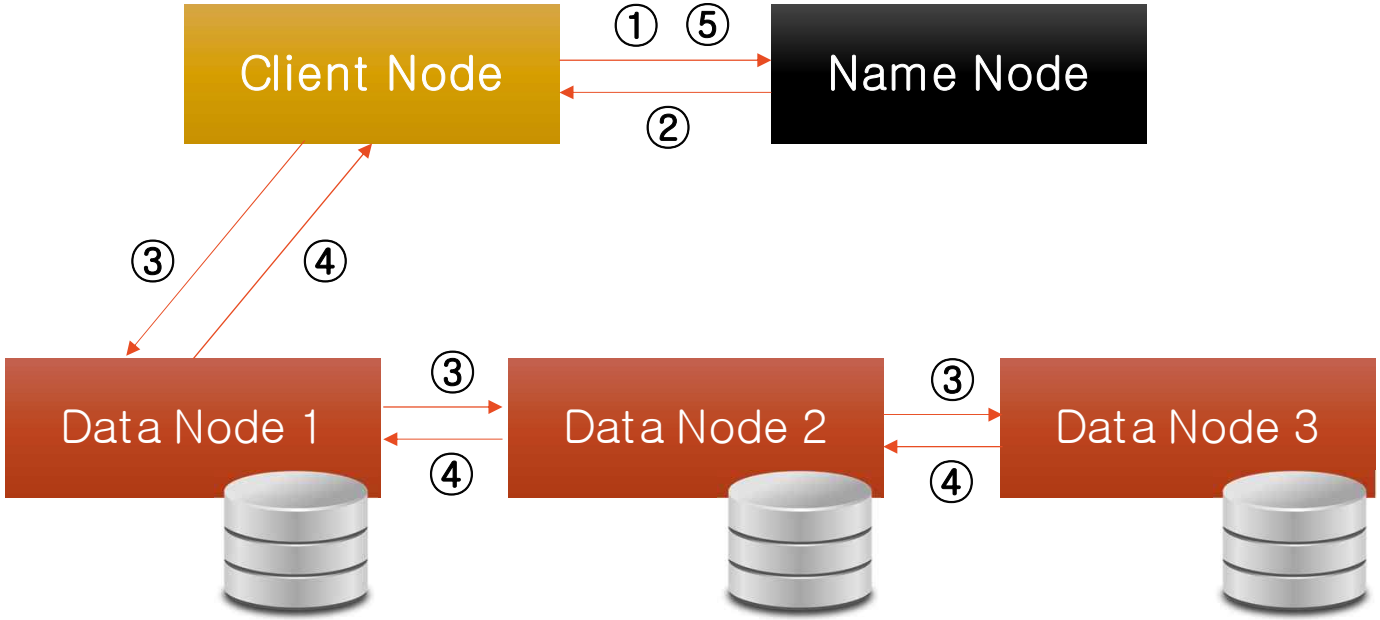
Reading a File

❖ 파일 읽기



Writing a File

❖ 파일 쓰기



Name Node Redilience

❖ Name node의 복원

- 메타데이터의 백업: 로그를 로컬 디스크와 NFS에 저장
- 2nd 네임노드: 주 네임노드의 로그 복사본을 유지
- HDFS federation: namespace volumne라는 서브디렉터리마다 분리된 name node를 지정함
- HDFS 고가용성
  - 공유 편집 로그(shared edit log)를 HDFS가 아닌 다른 안전한 공유 저장소에 저장함 (active name node)
  - Zookeeper는 active name node를 파악하고, 하나의 name node만 사용하도록 통제

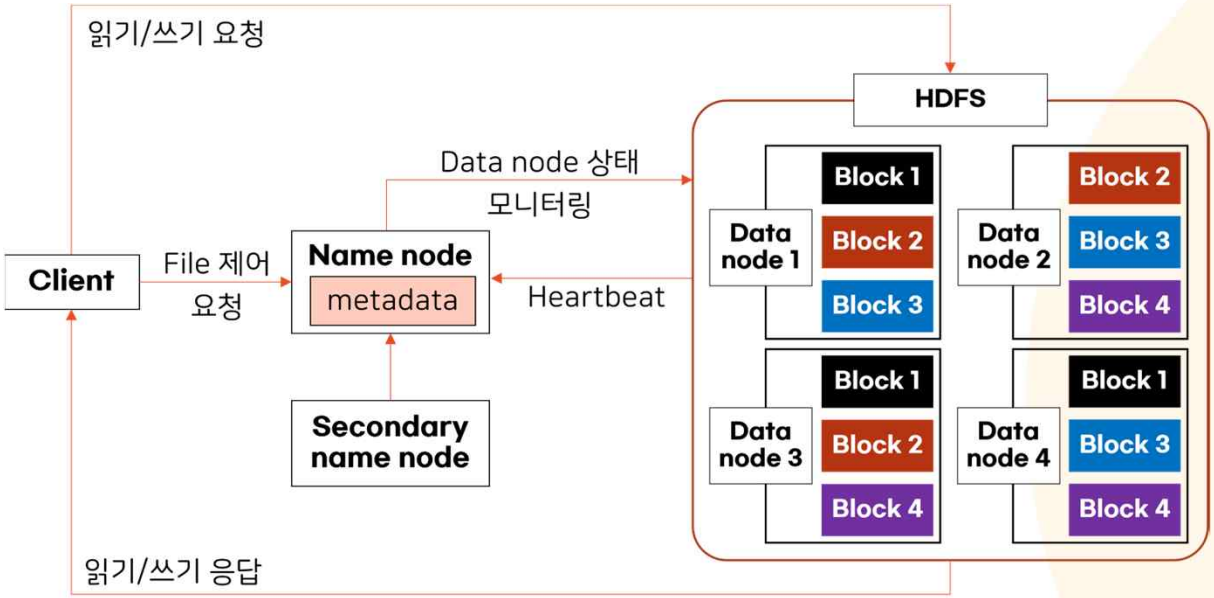
Using HDFS

❖ HDFS 사용법

- UI (Ambari)
- command-line interface
- HTTP / HDFS proxies
- Java interface
- NFS gateway

Summary

○ HDFS의 구조



MapReduce

-Definition and Process

Big Data

본 자료에는 네이버에서 제공한 나눔글꼴이 적용되어 있습니다.

Learning Objectives

❖ 학습내용

- MapReduce의 역할과 프로세스

❖ 학습목표

- MapReduce의 기능과 프로세스를 이해한다.

❖ MapReduce

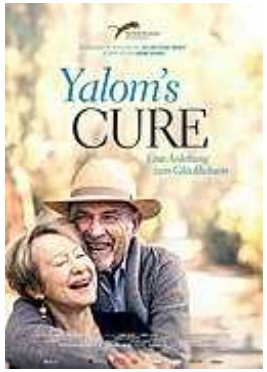
- 로직은 단순하나 입력 데이터의 크기가 매우 방대한 경우, 하나의 물리 머신에서 연산을 수행할 수 없는 상황을 해결할 수 있음
- 분산시켜 로직 수행 후, 결과를 하나로 합치는 아이디어
- Hadoop에 내재된 기능이며, 클러스터에 데이터의 처리를 분배함
- Mapper(transforming)과 Reducer(aggregating)으로 구성

MapReduce Example

❖ Example

Data set

User ID	Movie ID	Rating



MapReduce Example

❖ Example: Mapper

Input data

User ID Movie ID Rating

↓

Mapper

↓

Key 1 : value Key 2 : value Key 3 : value Key 1 : value · · ·

❖ Example: Mapper

Input data

User ID	Movie ID	Rating
001	335	4
002	127	5
013	781	3
217	453	3
002	334	3
001	265	4
001	478	5

Mapper

Key	Value
001	335
002	127
013	781
217	453
002	334
001	265
001	478

❖ Example: Mapper

Key	Value
001	335
002	127
013	781
217	453
002	334
001	265
001	478

Shuffle & Sort

001 :	335, 265, 478
002 :	127, 334
013 :	781
217 :	453

❖ Example: Reducer

001 :	335,265,478
002 :	127,334
013 :	781
217 :	453

Reducer

Value의 개수

001 :	3
002 :	2
013 :	1
217 :	1

Mapper and Reducer

❖ Process

Input data

User ID	Movie ID	Rating
001	335	4
002	127	5
013	781	3
217	453	3
002	334	3
001	265	4
001	478	5

Mapper and Reducer

❖ Process

Input data

User ID	Movie ID	Rating
001	335	4
002	127	5
013	781	3
217	453	3
002	334	3
001	265	4
001	478	5

Mapper

Key	Value
001	335
002	127

Key	Value
013	781
217	453
002	334

Key	Value
001	265
001	478

Mapper and Reducer

❖ Process

Key	Value
001	335
002	127

Key	Value
013	781
217	453
002	334

Key	Value
001	265
001	478

Shuffle & Sort

001 : 335,265,478
002 : 127,334

013 : 781
217 : 453

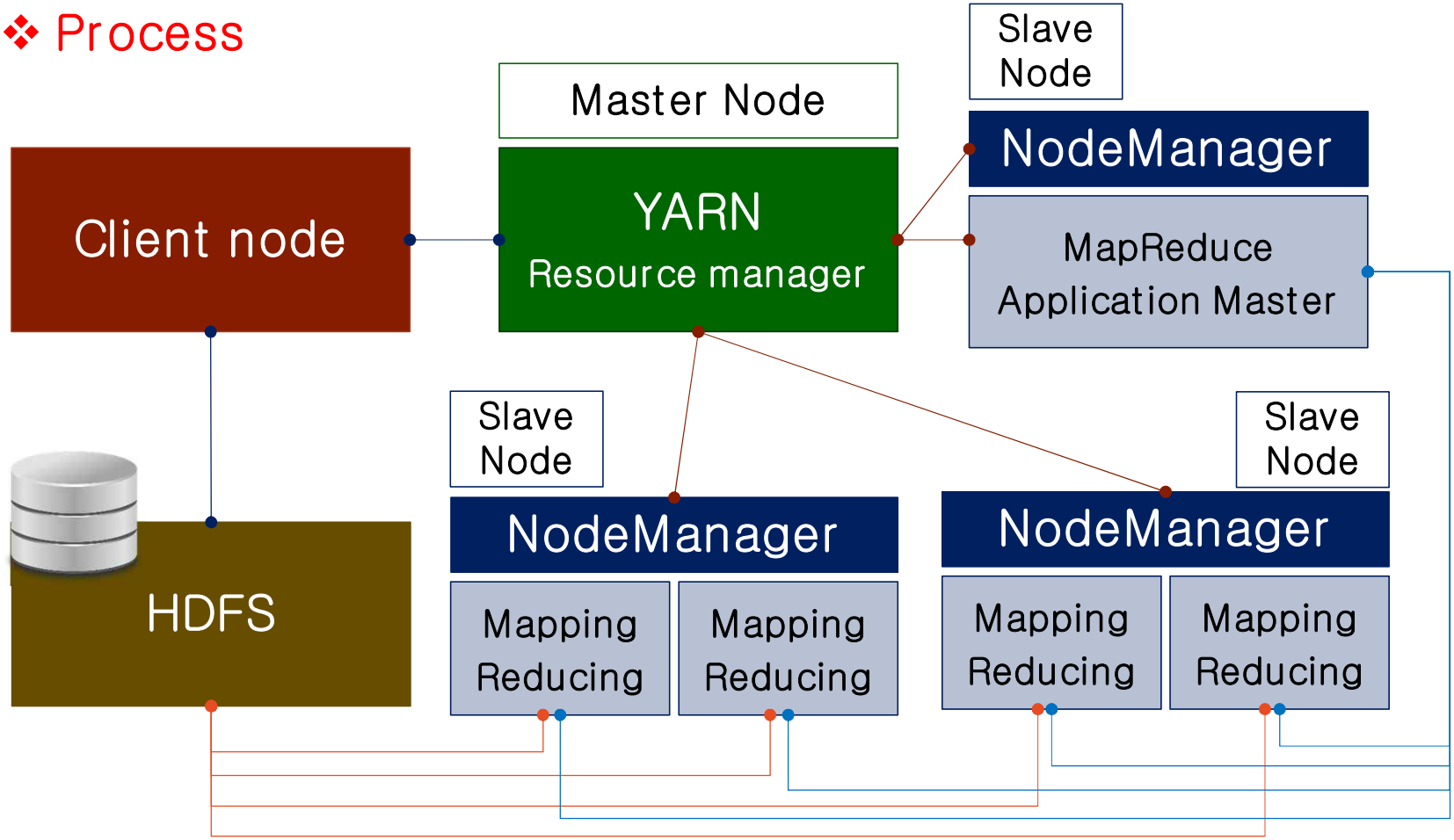
Reducer

001 : 3
002 : 2

013 : 1
217 : 2

Mapper and Reducer

❖ Process



❖ Handling failure

- Application master가 문제를 발견한 경우
  - 해당 작업을 재시작하거나, 다른 노드의 컨테이너가 해당 작업을 재시도함
- Application master 기능이 정상 동작하지 않을 경우
  - YARN resource manager가 가능한 노드에서 작업을 재시작
- YARN resource manager에 문제가 발생한 경우
  - Zookeeper를 이용하면 자동으로 백업 resource manager를 지정할 수 있음

Summary

- MapReduce
  - Mapper + Reducer

- MapRedue의 프로세스

Input data

User ID	Movie ID	Rating
001	335	4
002	127	5
013	781	3
217	453	3
002	334	3
001	265	4
001	478	5

Mapper

Key	Value
001	335
002	127
013	781
217	453
002	334
001	265
001	478

Shuffle & Sort

001 :	335, 265, 478
002 :	127, 334
013 :	781
217 :	453

Reducer

Value의 개수

001 :	3
002 :	2
013 :	1
217 :	1